

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code: CSA0305 Course Title: Data Structures
CO Assessed: CO5 - Naturalization (BL4, BL6) Assessment: Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage: 35% of CIA Total Marks: 100
CO5 Statement: Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.
Student Name: Sumitha Reg. No.: A2521097
Section / Batch: Date: 04/09/2026

Code of Conduct

I, Sumitha Sai S (A2521097) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sumitha

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

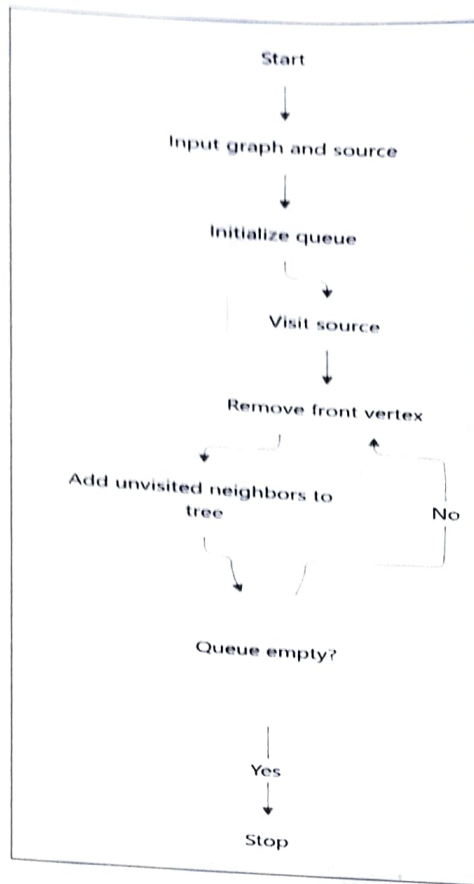
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Pseudocode Writing Task

1. Convert the flowchart for generating a BFS tree into code.

(20 Marks)

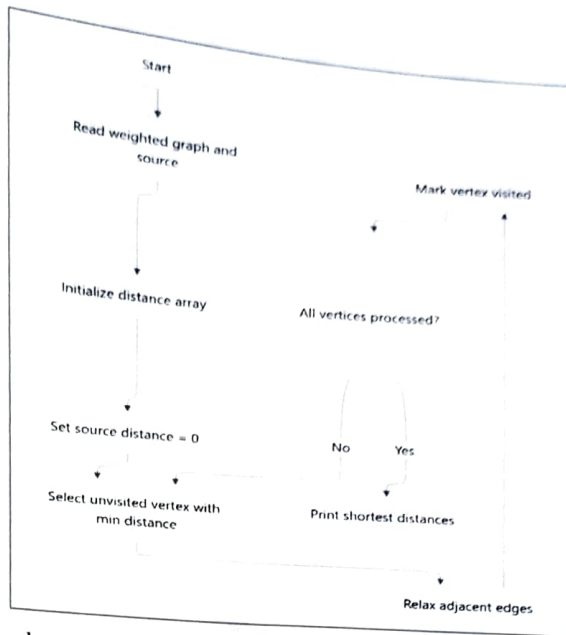
Flowchart Diagram:



2. Convert the flowchart for Dijkstra's shortest path algorithm into code.

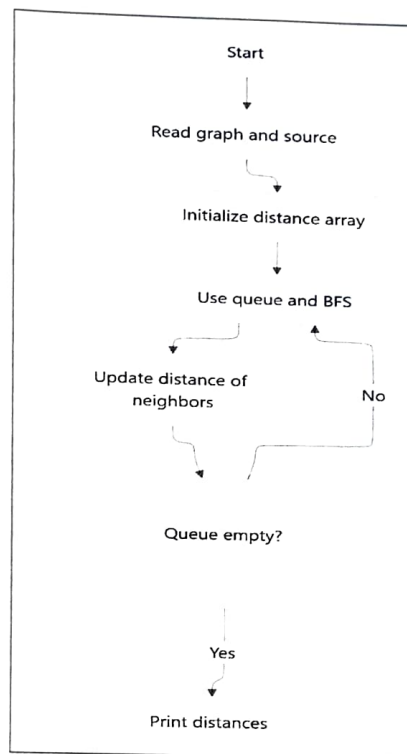
Flowchart Diagram:

(20 Marks)



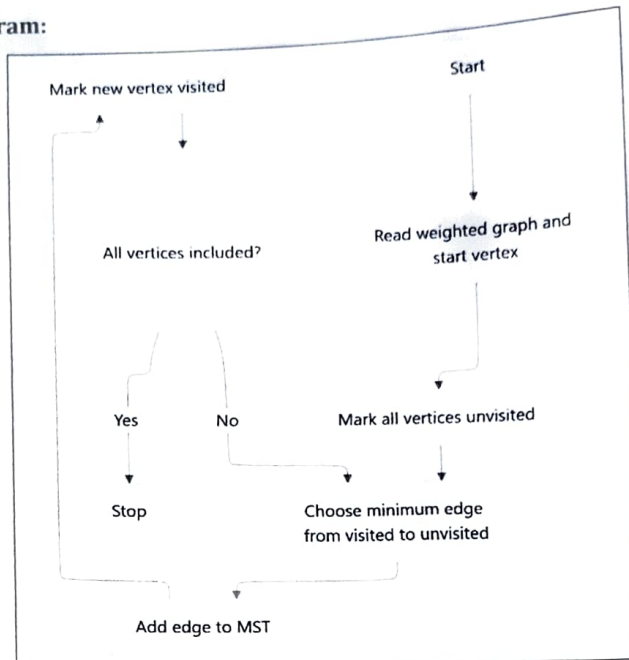
3. Convert the flowchart for shortest path in an unweighted graph into code.
Flowchart Diagram:

(20 Marks)



4. Convert the flowchart for Prim's algorithm into code.

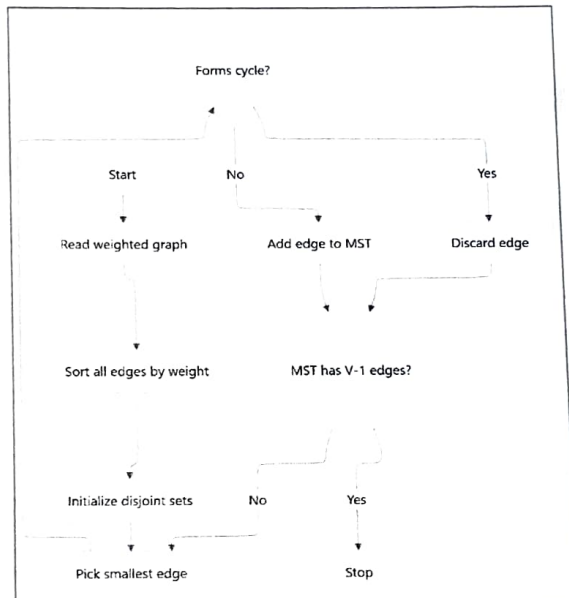
Flowchart Diagram:



5. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

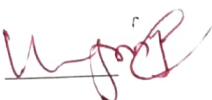
Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding, handles edge cases effectively	Good understanding, minor gaps in edge case handling	Partial understanding, misses some requirements	Misinterprets problem, major gaps
Code Correctness	30	Fully correct, passes all test cases	Mostly correct, minor errors	Partially correct, several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability, poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge

Signature: 

Date: _____

Course Coordinator

Signature: _____

Date: _____

Program Director

Signature: _____

Date: _____

BFS Tree generation

Aim: To generate a Breadth First Search (BFS) tree of a graph starting from a given source vertex.

Algorithm:

1. Read the no. of vertices and adjacency matrix
2. Read the starting vertex.
3. Initialize all vertices as unvisited
4. Mark the starting vertex as visited and insert into a queue
5. While the queue is not empty.
 - * Remove a vertex from the queue.
 - * Check all its adjacent vertices.
 - * If an adjacent vertex is unvisited.
 - ⇒ Mark it visited.
 - ⇒ Add the edge to the BFS tree.
 - ⇒ Insert the vertex into the queue.
6. Display the BFS traversal and BFS tree edges.

C program

```
#include <stdio.h>
#define N 10
int main () {
```

```

int a[N][N], q[N], vis[N] = {0};
int n, s, f = 0, r = 0, i, v;

scanf("%d", &n);
for (l = 0; i < n; i++)
    for (v = 0; v < n; v++)
        scanf("%d", &a[i][v]);

scanf("%d", &s);
q[r++] = s;
vis[s] = 1;

printf("BFS: ");
while (f < r) {
    int u = q[f++];
    printf("%d", u);

    for (v = 0; v < n; v++)
        if (a[u][v] && !vis[v]) {
            vis[v] = 1;
            q[r++] = v;
            printf("\n%d → %d", u, v);
        }
    return 0;
}

```

Sample Input:

No. of vertices: 5

Adjacency matrix:

0	1	0	0
1	0	0	1

```

1 0 0 0 1
0 1 0 0 1
0 0 1 1 0

```

(0 to 4) : 0

Sample Output:

BFS Traversal : 0 1 2 3 4

BFS Tree edges:

- 0 → 1
- 0 → 2
- 1 → 3
- 2 → 4

2. Dijkstra's Shortest Path

Aim:- To find the shortest path from a source vertex to all other vertices using Dijkstra's algorithm.

Algorithm:

1. Read the weighted adjacency matrix.
2. Initialize:
3. Select the unvisited vertex having the smallest distance.
4. Mark it as visited.
5. Relax all its adjacent vertices.
6. Repeat until all reachable vertices are processed.
7. Display.

C program:

```

#include <stdio.h>
#define N 10
#define INF 9999

```

```

int main() {
    int a[N][N], d[N], vis[N] = {0};
    int n, s, u, j, u, min;
    scanf("%d", &n);
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            scanf("%d", &a[i][j]);
    scanf("%d", &s);
    for (i=0; i<n; i++) d[i] = INF;
    d[s] = 0;
    for (i=0; i<n; i++) {
        min = INF; u = -1;
        for (j=0; j<n; j++)
            if (!vis[j] && d[i] < min)
                min = d[i], u = j;
        if (u == -1) break;
        vis[u] = 1;
        for (j=0; j<n; j++)
            if (a[u][j] && d[u] + a[u][j] < d[j])
                d[j] = d[u] + a[u][j];
    }
    for (i=0; i<n; i++)
        printf("%d \t", d[i]);
    return 0;
}

```

Shortest Path in an Unweighted Graph

Aim:-

To find the shortest path from a source vertex to all other vertices in an unweighted graph using BFS.

Algorithm:-

1. Read the adjacency matrix.
2. Set all distances to -1 .
3. Set source distance to 0 .
4. Insert source into the queue.
5. Remove a vertex from the queue.
6. Visit every unvisited adjacent vertex.

7. Set:

$$\text{distance}[\text{adjacent}] = \text{distance}[\text{current}] + 1.$$

C program:

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int main() {
```

```
    int graph a [N][N], d[N], q[N], vis[N] = {0};
```

```
    int n, s, f = 0, r = 0, i, v, u;
```

```
    scanf ("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (v = 0; v < n; v++)
```

```
            scanf ("%d", &a[i][v]);
```

```
scanf ("%d", &a[i][v]);
```

```
scanf ("%d", &es);
```

```
for (i=0; i<n; i++) d[i] = -1;
```

```
q[r++] = s;
```

```
vis[s] = 1;
```

```
d[s] = 0;
```

```
while (f < r) {
```

```
    u = q[f++];
```

```
    for (v=0; v<n; v++)
```

```
        if ((a[u][v] && !vis[v]) &
```

```
            vis[v] = 1;
```

```
            d[v] = d[u] + 1;
```

```
            q[r++] = v;
```

```
    }
```

```
}
```

```
for (i=0; i<n; i++)
```

```
    printf ("%d → %d = %d\n", s, i, d[i]);
```

```
return 0;
```

```
}
```

4. Prim's Algorithm -

Aim:

To construct a minimum spanning tree (MST) of a connected weighted graph using Prim's algorithm.

Algorithm:

1. Read the weighted adjacency matrix.
2. Initialize all vertices as not included in MST.
3. Set the key value of the first vertex to 0.
4. Select the vertex with minimum key value.
5. Include the MST.
6. Update the key values of its adjacent vertices.
7. Repeat until all vertices are included.
8. Display MST edges and total weight.

C program:

```
#include <stdio.h>
```

```
#define N 10
```

```
#define INF 9999
```

```
int main() {
```

```
    int a[N][N], key[N], vis[N] = {0}, par[N];
```

```
    int n, i, j, u, min, total = 0;
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < n; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
    for (i = 0; i < n; i++)
```

```
        key[i] = INF, par[i] = -1;
```

```

key[0] = 0;
for (i = 0; i < n; i++) {
    min = INF; u = -1;
    for (j = 0; j < n; j++)
        if (!vis[j] && key[j] < min)
            min = key[j], u = j;
    if (u == -1) break;
    vis[u] = 1;
    for (j = 0; j < n; j++)
        if (a[u][j] && !vis[j] && a[u][j] < key[j])
            key[j] = a[u][j], par[j] = u;
}
for (i = 1; i < n; i++) {
    printf("%d : %d\n", i, par[i]);
}
}

```

5. Kruskal's Algorithm.

Aim:

To construct a Minimum Spanning Tree (MST) of a connected weighted graph using Kruskal's algorithm.

Algorithm:

1. read the vertices and edges
2. store all edges with their weights.
3. Sort edges in ascending order of weight.
4. Initially, place every vertex in a separate set.
5. Select the smallest edge.
6. Check whether adding it creates a cycle.

C program.

```
#include <stdio.h>
```

```
#define MAX 100
```

```
struct Ed
```

```
{ int u, v, w;
```

```
};
```

```
int p[N];
```

```
int find (int x) {
```

```
    return p[x] == x ? x : (p[x] = find (p[x]));
```

```
}
```

```
int main() {
```

```
    int n, m, i, j, c = 0, total = 0;
```

```
    scanf ("%d %d", &n, &m);
```

```
    for (i = 0; i < m; i++)
```

```
        scanf ("%d %d %d", &e[i].u, &e[i].v, &e[i].w);
```

```
    for (i = 0; i < m - 1; i++)
```

```
        for (j = 0; j < m - i - 1; j++)
```

```
            if (e[j].w > e[j + i].w)
```

$t = e[j], e[j] = e[j+1], e[j+1] = t;$

for ($i=0; i < n; i++$) $p[i] = i;$

for ($i=0; i < m$ & $e < n-1; i++$) {

int $a = \text{find}(e[i].u), b = \text{find}(e[i].v);$

if ($a \neq b$) {

printf (" $\%d - \%d : \%d \setminus n$ "),

$e[i].u, e[i].v, e[i].w$);

$p[a] = b;$

$\text{total} += e[i].w;$

}
}
}

WGP